

webcrawl — Feature Inventory & Parity Contract

This document is the **zero-function-loss contract** for the unified-workspace rebuild. Everything listed here existed before the rebuild and **must keep working** (additively). The rebuild adds new capabilities on top; it does not remove or degrade anything below.

Legend:  preserved unchanged ·  extended (back-compatible) ·  added by the rebuild.

1. HTTP API routes (`src/server.js`)

Method	Route	Status	Notes
POST	/v1/scrape	✓	URL → markdown/ html/rawHtml/links/ screenshot
POST	/v1/map	✓	site URL discovery
POST	/v1/search	✓	keyless DuckDuckGo search
POST	/v1/extract	✓	URL + schema → structured JSON (LLM or heuristic)
POST	/v1/crawl	✓	async crawl job (wait for sync)
GET	/v1/crawl/:id	✓	crawl job status
DELETE	/v1/crawl/:id	✓	advisory cancel
GET	/v1/test	✓	module health + chromium + llm reachability
POST	/v1/research	✓	intent-first research job (async)
GET	/v1/research/:id	✓	poll research job (? since= for cheap deltas)
GET	/v1/research/:id/ stream	✓	SSE stage stream
POST	/v1/research/schema	✓	plain-English → JSON schema
GET	/v1/sessions	✓	list saved sessions
GET	/v1/sessions/com- pare	✓	compare two ses- sions
GET	/v1/sessions/:id	✓	full session
POST	/v1/sessions/:id/re- run	✓	rerun a saved session

Method	Route	Status	Notes
GET	/v1/mcp	✓	MCP bootstrap (tools+hosts+defaults)
GET	/v1/mcp/tools	✓	MCP tool enable/disable state
POST	/v1/mcp/tools	✓	toggle MCP tools
GET	/v1/mcp/config	✓	generate a host client config
GET	/v1/llm	✓	current LLM settings + presets
POST	/v1/llm	✓	apply LLM settings live
GET	/v1/llm/models	✓	list endpoint model ids
GET	/v1/localfs	NEW	list approved allowlist roots
POST	/v1/localfs/roots	NEW	add an allowlisted root (read-only)
DELETE	/v1/localfs/roots	NEW	remove an allowlisted root
POST	/v1/localfs/search	NEW	search filenames + contents within allowlist
GET	/v1/localfs/file	NEW	read one allowlisted file (read-only)
GET	/v1/mcp/servers	NEW	list MCP servers (registry)
POST	/v1/mcp/servers	NEW	add/update an MCP server
DELETE	/v1/mcp/servers/:id	NEW	remove an MCP server
POST		NEW	

Method	Route	Status	Notes
	/v1/mcp/servers/:id/ toggle		enable/disable one server
POST	/v1/mcp/servers/ enable-all	NEW	enable/disable all servers
POST	/v1/mcp/servers/:id/ test	NEW	test-connection (static validation)
POST	/v1/mcp/import	NEW	parse+validate+preview pasted MCP JSON
GET	/v1/mcp/templates	NEW	starter MCP server templates
GET	/v1/mcp/profiles	NEW	list MCP profiles
POST	/v1/mcp/profiles	NEW	create/save a profile
POST	/v1/mcp/profiles/:id/apply	NEW	apply a profile (enable its servers)
DELETE	/v1/mcp/profiles/:id	NEW	remove a profile
POST	/v1/agent	NEW	run the agentic pipeline (modes+limits+routing)
GET	/v1/agent/:id	NEW	poll an agent run
GET	/v1/workspaces	NEW	list workspaces (superset of sessions)
GET	/v1/workspaces/:id	NEW	full workspace
POST	/v1/workspaces/:id/ duplicate	NEW	duplicate a workspace
GET	/v1/workspaces/:id/ export	NEW	export a workspace as JSON

2. MCP stdio server (`src/mcp.js`) — 6 tools (unchanged, backward-compatible)

`webcrawl_scrape` , `webcrawl_crawl` , `webcrawl_map` , `webcrawl_search` , `webcrawl_extract` , `web-crawl_screenshot` .

Enable/disable persisted in `data/mcp-settings.json` , honored at startup.

3. Core engines (`src/core/`)

Module	Status	Purpose
<code>scrape.js</code>	✓	fetch/browser scrape → markdown (+ <code>fetchMethod</code> provenance)
<code>crawl.js</code>	✓	same-origin BFS crawl
<code>search.js</code>	✓	keyless DuckDuckGo search
<code>map.js</code>	✓	site URL discovery
<code>extract.js</code>	✓	schema extraction (LLM/heuristic)
<code>fetch.js</code>	✓	HTTP fetch + href helpers
<code>browser.js</code>	✓	Playwright headless chromium
<code>markdown.js</code>	✓	HTML → markdown
<code>config.js</code>	✓	server config (port/concurrency/timeouts)
<code>llm.js</code>	✓	selectable providers (anthropic/openai/cloud/Imstudio/ollama/llamacpp)
<code>research.js</code>	+	intent-first planner + <code>runResearch</code> (now source-aware via agent)
<code>sessions.js</code>	✓	JSON-file session store (<code>data/sessions/</code>)
<code>mcp-config.js</code>	✓	MCP tool metadata + client-config builders
<code>localfs.js</code>	NEW	allowlisted, read-only local filesystem search
<code>mcp-servers.js</code>	NEW	MCP server registry + profiles + import/validate + templates
<code>agent.js</code>	NEW	agentic execution (modes/limits/routing) over the core engines

Module	Status	Purpose
<code>workspaces.js</code>		workspaces layer (duplicate/export) over sessions

4. WebUI controls (`webui/index.html`) — all preserved

Simple (intent-first) mode , Advanced mode with direct Scrape/Search/Crawl/Map/Extract panels , Model panel , Sessions drawer , History , Module Health , in-WebUI MCP tool panel .

 added: theme toggle (vibrant dark + clean light), multi-source selector, research-domain + depth presets, agent-mode selector + limits, MCP Manager (servers/profiles/import/templates), results workspace tabs (Answer/Sources/Data/Raw/Files/Agent Activity/Tool Calls/Logs), saved-workspace actions (duplicate/export), local-filesystem panel.

5. Persisted settings / runtime state (all git-ignored under `data/`)

Key	Status	File
MCP tool enable/disable		<code>data/mcp-settings.json</code>
Saved research sessions		<code>data/sessions/*.json</code>
LLM settings		in-memory (seeded from env), patched via <code>POST /v1/llm</code>
MCP server registry		<code>data/mcp-servers.json</code>
MCP profiles		<code>data/mcp-profiles.json</code>
Local-FS allowlist		<code>data/localfs-allowlist.json</code>
Theme preference		browser <code>localStorage</code>

6. Security invariants (must never regress)

1. Local filesystem access is **read-only** and **strictly confined to the approved allowlist**; path traversal is rejected server-side.
2. Scraped/retrieved/imported content is **untrusted** — fenced before reaching any LLM; it can never alter the plan, permissions, or tool set (prompt-injection defense).
3. No real secrets are committed or echoed; MCP import/config use `sk-ant-...`-style placeholders.
4. Destructive / write-capable actions require explicit approval (agent write tools are opt-in).
5. The app runs fully offline of Abacus: local LLM or user key, keyless search — no cloud dependency.